

New Generation Data Models and DBMSS

Project: Credit Card Fraud Detection

January 2026

Sofia Maggioni 75788A
sofia.maggioni1@studenti.unimi.it

Cecilia Zombini 60139A
cecilia.zombini@studenti.unimi.it

Contents

1 Introduction **3**
- 1.1 Project overview . . . 3
- 1.2 Objectives . . . 3

2 Conceptual design **5**
- 2.1 UML Class Diagram . . . 5
- 2.2 Assumptions and Constraints . . . 6

3 Logical Data Model **7**
- 3.1 Choice of Neo4j . . . 7
- 3.2 Class → Node Mapping . . . 8
- 3.3 Association → Edge Mapping . . . 8
- 3.4 Model Motivations . . . 9

4 Data Loading Description **10**
- 4.1 Script Description . . . 10
  - 4.1.1 Data Generation (*Generate_dataset.py*) . . . 10
  - 4.1.2 Data Loading (*Neo4j_populate.py*) . . . 10
- 4.2 Handling the 3 Dataset . . . 11

5 Implementation of Required Operations **12**
- 5.1 Data Enrichment . . . 12
  - 5.1.1 Logical Model Extension . . . 12
  - 5.1.2 Extending Transactions . . . 13
  - 5.1.3 Identifying Frequent Collaborators . . . 13
- 5.2 Identifying Customers with Similar Habits (Query A) . . . 14
- 5.3 Outlier Detection via Temporal Analysis (Query B) . . . 16
- 5.4 Co-Customer Network Analysis (Query C) . . . 17
  - 5.4.1 Standard Implementation (Small Medium Datasets) . . . 17
  - 5.4.2 Optimized Implementation (Large Dataset) . . . 18
- 5.5 Weekly Outlier Analysis (Query E) . . . 19

6 Performance Analysis **22**
- 6.1 Dataset Generation . . . 23
- 6.2 Neo4j Population . . . 24
- 6.3 Update Transactions (d.i) . . . 25
- 6.4 Create Collaborators (d.ii) . . . 26
- 6.5 Query A . . . 27
- 6.6 Query B . . . 29
- 6.7 Query C . . . 31
- 6.8 Query E . . . 33

7 Conclusions **35**

1 Introduction

1.1 Project overview

This project focuses on the design, implementation, and evaluation of a NoSQL database system tailored for the domain of Credit Card Fraud Detection. While payment card fraud represents a significant financial challenge requiring sophisticated detection methods, this initiative does not focus on the implementation or validation of Machine Learning (ML) algorithms. Instead, the primary scope is to engineer a robust data storage and retrieval solution that supports the data-intensive nature of fraud analysis.

To achieve this, the project utilizes a transaction data simulator to generate synthetic datasets representing Customer profiles, Terminal profiles, and Transactions. The core challenge lies in modeling this data within a NoSQL environment (selecting from MongoDB, Neo4J, or Oracle PGQL) to optimize complex analytical operations, such as identifying co-customer networks and detecting potential outliers. Furthermore, the project addresses the flexibility of NoSQL systems by extending the logical model with additional attributes (e.g., payment methods, customer satisfaction) and relationships after the initial data load.

1.2 Objectives

The main goal of this project is to deliver a high-performance NoSQL data layer capable of handling increasing volumes of transaction data. The specific objectives are as follows:

- **Dataset Generation and Scaling**: to generate three distinct synthetic datasets of increasing size (minimum 50MB, 100MB, and 200MB) containing at least 1000 customers and 200 terminals, ensuring the system's ability to handle data scalability.
- **Data Modeling and Optimization**: to define a conceptual model and a subsequent logical data model specifically designed to optimize the execution of complex queries, such as identifying shared terminal usage among customers and calculating transaction outliers.
- **Complex Query Implementation**: to develop and implement scripts that execute sophisticated analytical operations, including the computation of "co-customer networks" and the analysis of fraudulent patterns across different timeframes.
- **Schema Evolution**: to demonstrate the flexibility of the chosen NoSQL system by extending the database schema post-deployment to include new transaction details (e.g., promotional offers, satisfaction ratings) and defining new queryable relationships like "frequent collaborators".

- **Performance Evaluation**: to rigorously measure and analyze the execution times for data loading, updating, and querying across all three dataset sizes, providing a discussion on performance trends and potential optimization patterns.

2 Conceptual design

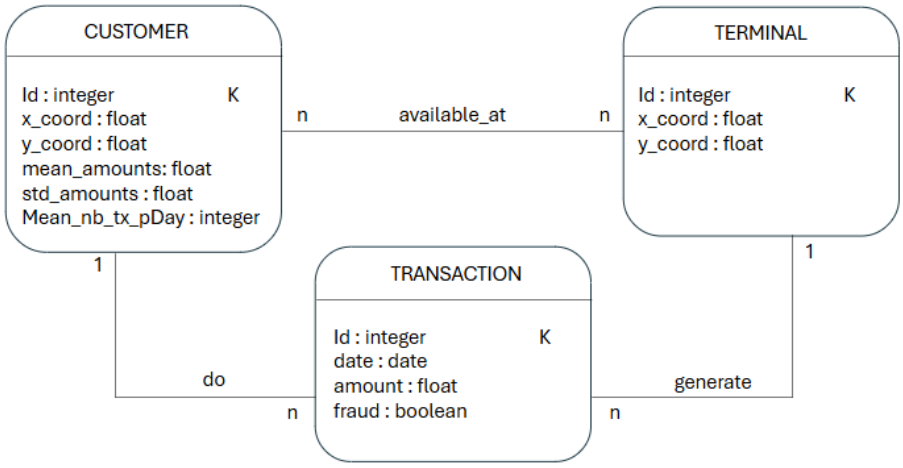


Figure 1: UML Class Diagram

2.1 UML Class Diagram

The proposed conceptual model, illustrated in Figure 1, is designed to capture the essential entities involved in the credit card fraud detection domain: Customer, Terminal, and resulting Transaction. This model serves as the blueprint for the subsequent NoSQL implementation. Here **entities** and their **attributes** are explained:

- **Customer**: this entity represents the cardholder. It is uniquely identified by an *Id* (Integer). To facilitate fraud analysis, the customer profile includes spatial data (*x_coord, y_coord*) representing his geographical location, and behavioral metrics derived from historical activity:
  - *mean_amounts* (Float): the average value of the customer's transactions.
  - *std_amounts* (Float): the standard deviation of transaction amounts, useful for detecting anomalies.
  - *Mean_nb_tx_pDay* (Integer): the average number of transactions the customer performs daily.
- **Terminal**: this entity represents the point of sale (POS) or ATM. It is uniquely identified by an *Id* (Integer) and characterizes the physical location of the merchant using *x_coord* and *y_coord* (Float).

- **Transaction**: this is the central entity linking customers and terminals. It represents a single payment event identified by a unique *Id* (Integer). Its attributes include:
  - *date*: the timestamp of the operation.
  - *amount* (Float): the monetary value paid.
  - *fraud* (Boolean): a flag indicating whether the transaction is legitimate (0/False) or fraudulent (1/True).

The diagram defines also the following **relationships** between entities:

- **Customer – Transaction ("do")**: this is a **1-to-N** (One-to-Many) relationship. A single customer can perform multiple transactions over time, but each specific transaction instance belongs to exactly one customer.
- **Terminal – Transaction ("generate")**: this is a **1-to-N** (One-to-Many) relationship. A terminal generates many transactions, but each transaction occurs at a single, distinct terminal.
- **Customer – Terminal ("available_at")**: this is a **N-to-N** (Many-to-Many) relationship. It models the habit or availability profile of the user, indicating the set of terminals where a customer typically performs transactions or is geographically located near. This pre-association helps in identifying anomalies if a transaction occurs at a terminal not linked to the customer's usual profile.

2.2 Assumptions and Constraints

To ensure the model accurately reflects the domain and the synthetic data generation process, the following assumptions and constraints are applied:

- **Primary Key (K)**: the *Id* attribute in all three classes (Customer, Terminal, Transaction) serves as the primary key and must be unique across the respective datasets. Also, every transaction must be associated with one valid Customer Id and one valid Terminal Id. Orphaned transactions are not permitted.
- **Spatial Assumption**: the coordinates (*x_coord*, *y_coord*) are assumed to be on a consistent Euclidean plane, allowing for distance calculations between customers and terminals to assess the likelihood of fraud. For this reason, they are set as Float (and not Integer) to match the type of real coordinates.
- **Fraud Flag**: the fraud attribute is a boolean constraint, strictly limiting the classification of a transaction to two states: fraudulent or valid.

3 Logical Data Model

3.1 Choice of Neo4j

The decision to implement this project using Neo4j, a native graph database, is driven by the highly interconnected nature of the domain and the specific requirements of the analytical queries defined in the project scope.

While the dataset involves structured entities (Customers, Terminals, Transactions), the complexity lies in the relationships rather than the isolated data points. Analyzing the required operations reveals why a graph model is superior to relational or document-oriented alternatives.

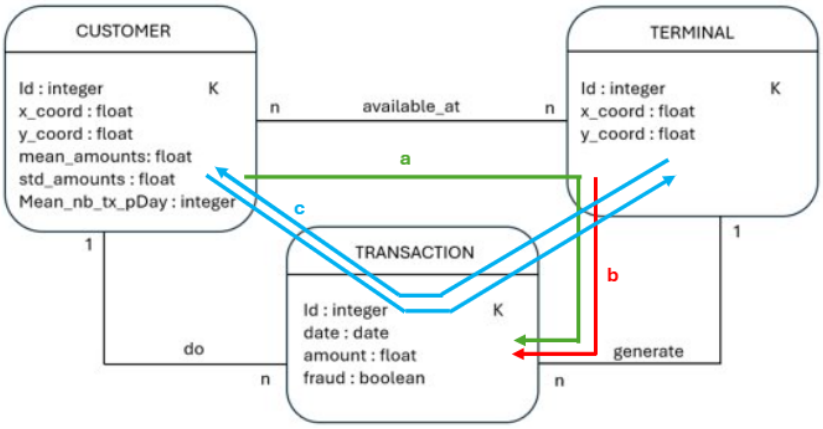


Figure 2: UML Class Diagram with workflow analysis

As highlighted in our query analysis in Figure 2, the most critical operations involve deep traversals. Specifically, Query C (Co-customer network) and Query A (Shared Terminals) require navigating a path from a Customer to a Transaction, then to a Terminal, and potentially back to other Customers. In the provided analysis diagram, the **light blue arrow** represents this multi-hop traversal ($Customer \rightarrow Transaction \rightarrow Terminal \ldots$). In a Relational Database, this would require computationally expensive JOIN operations (e.g., joining the Transaction table to itself via Terminals). In Neo4j, this is a native path traversal, which is significantly faster and syntactically simpler (using Cypher).

The requirements also ask to identify "collaborators" or clusters of users. Graph databases excel at these "friends-of-friends" style queries, allowing us to hop through the network efficiently without scanning the entire dataset.

Neo4j allows for a schema-flexible approach, which supports the requirement to extend the model later (e.g., adding "Promotional Offer" attributes or "Frequent Collaborator" relationships) without altering the existing structure.

The proposed Graph Logical Model is based based on the Conceptual UML and is shown in Figure 3.

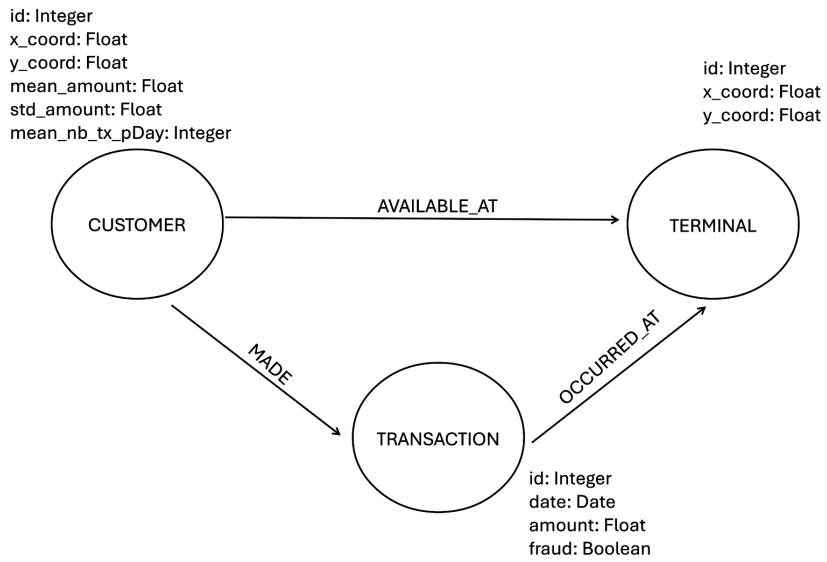


Figure 3: Graph Logical Model

3.2 Class → Node Mapping

The Logical Graph Model maps the primary classes directly to Nodes (vertices) with specific Labels. The proposed graph model consists of three primary entities: Customer, Terminal, and Transaction.

- The **Customer** Class, labeled :CUSTOMER, represents individuals using the system and includes properties such as *id* (Integer) for unique identification, *x_coord* and *y_coord* (Float) for spatial location, and transaction-related metrics like *mean_amount* (Float), *std_amount* (Float), and *mean_nb_tx_pDay* (Integer) that capture the average transaction behavior.
- The **Terminal** class, labeled :TERMINAL, represents physical points of sale and includes *id* (Integer) and spatial coordinates *x_coord* and *y_coord* (Float).
- The **Transaction** class, labeled :TRANSACTION, represents individual operations between customers and terminals, with properties including *id* (Integer), *date* (Date), *amount* (Float), and a *fraud* flag (Boolean) indicating whether the transaction is fraudulent.

3.3 Association → Edge Mapping

The associations defined in the UML are transformed into directed Relationships (Edges) in Neo4j. The proposed graph model defines three key relationships connecting the entities: [:MADE], [:OCCURRED_AT], [:AVAILABLE_AT].

- **[:MADE]** relationship, directed from (:CUSTOMER) to (:TRANSACTION), indicates that a specific customer initiated a particular transaction.
- **[:OCCURRED_AT]** relationship, directed from (:TRANSACTION) to (:TERMINAL), links each transaction to the physical terminal where it took place, providing spatial context for the event.
- **[:AVAILABLE_AT]** relationship, directed from (:CUSTOMER) to (:TERMINAL), captures the customer's habitual usage patterns by connecting them to terminals they are expected to use, helping to model normal behavior for anomaly detection.

3.4 Model Motivations

The proposed graph modelin Figure 3 is designed to optimize both write performance and query expressiveness.

One might consider modeling a transaction simply as an edge between a Customer and a Terminal (e.g., (:Customer)-[:PAID]→(:Terminal)). However, we chose to model Transaction as a distinct Node. This is crucial because transactions have their own rich properties (Amount, Date, Fraud status) that are often the target of filtering. It allows the "Light Blue Path" in Figure 2 to explicitly step through the transaction event ($Customer \rightarrow Transaction \rightarrow Terminal$), which is essential for auditability and for Query A (comparing transaction counts).

We maintained the direct link between Customer and Terminal. This allows for rapid comparison between where a transaction did occur (via the OCCURRED_AT path) and where the customer usually goes (via the AVAILABLE_AT path), simplifying anomaly detection. By using directed relationships, we reduce the search space for queries. For example, to find a customer's spending history, the engine traverses only outgoing [:MADE] relationships, ignoring incoming paths.

4 Data Loading Description

This section details the technical approach used to generate the synthetic data and the procedure implemented to populate the NoSQL database. The process is divided into two distinct phases: the generation of CSV synthetic data and the subsequent batch loading into Neo4j.

4.1 Script Description

The implementation relies on two main Python scripts: *Generate_dataset.py* for data creation and *Neo4j_populate.py* for database ingestion.

4.1.1 Data Generation (*Generate_dataset.py*)

This script is responsible for creating the synthetic datasets. It is built upon the Transaction Data Simulator code provided in Chapter 3 of the reference book, *Machine Learning for Credit Card Fraud Detection - Practical Handbook1* available at this_link.

We integrated the handbook's core functions (*generate_customer_profiles_table*, *generate_transactions_table*, and *add_frauds*) into a custom orchestration function (*generate_project_datasets*). This allows us to:

- **Generate Entities**: create consistent Customer and Terminal profiles with geographical coordinates and behavioral properties.
- **Simulate Transactions**: generate transactions based on customer spending habits (mean amount, frequency).
- **Inject Fraud**: apply the three specific fraud scenarios defined in the handbook.
- **Output**: the script exports five separate CSV files (*customer_profiles.csv*, *terminal_profiles.csv*, *dataset_small_50MB.csv*, *dataset_medium_100MB.csv* and *dataset_large_2000MB.csv*) into an *output_datasets* folder.

4.1.2 Data Loading (*Neo4j_populate.py*)

This script manages the connection to the Neo4j instance via the official Bolt driver. To ensure efficiency and data integrity, the script performs the following steps:

- **Batch Processing**: instead of inserting rows one by one, the script utilizes the Cypher UNWIND clause combined with batch processing (defaulting to 5,000 records per batch). This minimizes network overhead and memory consumption during the transaction load.
- **Data Cleaning**: it includes a *clear_database()* function to wipe existing nodes and relationships, ensuring an idempotent and clean state for every test run.

- **Safe Parsing**: custom logic was implemented to safely parse the *available_terminals* list string from the CSV, handling potential formatting issues during the import of customer profiles.

4.2 Handling the 3 Dataset

To fulfill the project requirement of evaluating performance across datasets of increasing size (minimum 50MB, 100MB, and 200MB), we adopted a "time-window expansion" strategy. Since the number of Customers (5,000) and Terminals (10,000) remains static to maintain a consistent network density, we scaled the datasets by increasing the simulation duration (nb_days). This results in a proportional increase in the number of transactions and overall file size. The generation strategy is summarized in Table 1.

Dataset	**Target Size**	**Simulation Duration**
Small	∼50 MB	97 Days
Medium	∼100 MB	187 Days
Large	∼200 MB	367 Days

Table 1: Dataset size and simulation duration

The loading script *Neo4j_populate.py* is designed with a configuration variable (*SCENARIO_TARGET*) that allows the user to select which of these three datasets to load ("SMALL", "MEDIUM", or "LARGE"). The script automatically maps the selection to the correct CSV file and tracks the execution time for loading Customers, Terminals, and Transactions separately, providing the metrics necessary for the performance evaluation discussed in the Chapter 6.

5 Implementation of Required Operations

Experimental Note: for all the operations described in the following subsections (Data Enrichment, Query A, B, C, E), the execution protocol remained consistent. Each Python script was executed three distinct times, targeting the Small (50MB), Medium (100MB), and Large (200MB) datasets respectively.

5.1 Data Enrichment

As specified in the project requirements (Point d), the database schema was designed to be flexible. After the initial loading of the core dataset (Customers, Terminals, Transactions), we implemented a schema evolution phase to enrich the data with additional business metrics and social relationships. This phase was executed using two dedicated Python scripts to ensure modularity and performance.

5.1.1 Logical Model Extension

The logical graph model was updated to accommodate these new requirements, as illustrated in the following Figure 4.

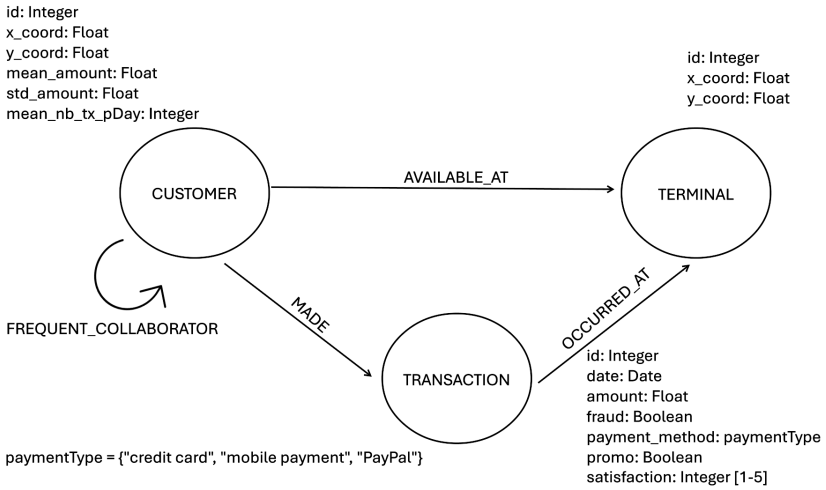


Figure 4: Extended Graph Logical Model

The specific changes are:

- **Transaction Node Extension**: the Transaction entity, which initially contained only technical data (Amount, Date, Fraud), was enriched with qualitative attributes:
  - *payment_method*: a categorical string (e.g., "Credit Card", "PayPal").

  - *promo*: a boolean flag indicating if the transaction was part of a promotion.
  - *satisfaction*: an integer rating (1–5) representing customer feedback.

- **New relationship FREQUENT_COLLABORATOR**: a new **N-to-N** (Many-to-Many) recursive relationship was defined on the Customer node. This connects customers who exhibit similar behavioral patterns at the same locations.

5.1.2 Extending Transactions

To populate the new properties on millions of existing transaction nodes, we developed the *Neo4j_update_transactions.py* script. Since the original dataset did not contain this information, the script synthetically generates it directly within the database using Cypher's server-side capabilities.

- **Random Assignment Strategy**: instead of generating data in Python and sending it over the network, the script uses a Cypher query with rand() functions to assign values efficiently:
  - **Payment Method**: assigned randomly with equal probability among "Credit Card", "Mobile Payment", and "PayPal".
  - **Promotional Offer**: assigned True with a 15% probability (rand() < 0.15).
  - **Satisfaction**: assigned an integer between 1 and 5.
- **Batch Processing**: the script iterates through the database in batches (default size: 10,000) using a LIMIT clause on transactions where *payment_method* IS NULL. This approach prevents memory saturation on the Neo4j heap during the update of the 200MB dataset.

5.1.3 Identifying Frequent Collaborators

The second requirement was to connect customers who are "Frequent Collaborators." A collaborator pair is defined as two customers who:

1. have made at least 5 transactions at the same terminal.
2. have an average satisfaction rating within 0.5 of each other.

This operation is computationally intensive. To optimize execution, we implemented a two-step algorithm in *Neo4j_create_collaborators.py*:

- Step 1: Pre-calculation of Averages. Calculating the average satisfaction for every comparison would be redundant and slow. The script first executes a global aggregation query that computes avg(tx.satisfaction) for every customer and persists this value as a new property, *c.avg_satisfaction*, on the Customer node.

- Step 2: Relationship Creation. The script then processes the terminals one by one (to avoid transaction timeouts). For each terminal ID:
    1. It retrieves all customers who have performed ≥ 5 transactions at that specific terminal.
    2. It creates a collection of these "frequent users".
    3. It performs a Cartesian product (User A vs. User B) within the collection.
    4. It applies the filter abs(UserA.avg - UserB.avg) ≤ 0.5.
    5. It creates the [:FREQUENT_COLLABORATOR] relationship between matching pairs using MERGE to ensure no duplicates are created.

This "Pre-calculation + Batching" strategy ensures that the complex relationship logic remains queryable and performant even on the largest dataset.

5.2 Identifying Customers with Similar Habits (Query A)

The goal of this operation is to identify pairs of customers (M and N) who exhibit a strong behavioral correlation. Specifically, the system must return pairs that satisfy two simultaneous conditions:

1. Spatial Overlap: they have transacted at at least 4 of the same terminals.
2. Activity Similarity: their total transaction history count differs by no more than 2.

This query leverages the graph structure to traverse the path from Customer M to Customer N via shared Terminals. Instead of performing expensive table joins as in a relational database, Neo4j navigates the pattern (Customer)-[:PERFORMED]→(Transaction)-[:OCCURRED_AT]→(Terminal). To ensure optimal performance, the query uses a "filter-early" strategy. We first filter pairs based on the shared terminals (Condition 1) before calculating their total transaction counts (Condition 2), as the latter is a more expensive operation involving the entire history of the customers. The following Cypher 5.2 statement was implemented in the script *QueryA.py*:

```
MATCH (m:Customer)-[:PERFORMED]->(:Transaction)-[:
    OCCURRED_AT]->(t:Terminal)
      <-[:OCCURRED_AT]-(:Transaction)<-[:PERFORMED]-(n:
          Customer)

WHERE m.id < n.id

WITH m, n, count(DISTINCT t) as shared_terminals
WHERE shared_terminals >= 4

MATCH (m)-[:PERFORMED]->(txM:Transaction)
WITH m, n, shared_terminals, count(txM) as countM

MATCH (n)-[:PERFORMED]->(txN:Transaction)
WITH m, n, shared_terminals, countM, count(txN) as countN

WHERE abs(countM - countN) <= 2

RETURN
    m.id as Customer_M,
    n.id as Customer_N,
    shared_terminals as Shared_Terminals,
    countM as Tx_Count_M,
    countN as Tx_Count_N
ORDER BY shared_terminals DESC
```

Listing 1: Cypher query A for identifying customers sharing terminals

Detailed Step-by-Step explanation:
Pattern Matching (MATCH): the query starts by finding a "bridge" pattern: two customers (M and N) are connected if they both have a path to the same Terminal node t.
Symmetry Filter (m.id < n.id): since the relationship "is similar to" is symmetric (if A is similar to B, B is similar to A), we enforce an ID order. This prevents duplicate results and, crucially, halves the computational search space.
Intermediate Aggregation (WITH ... count(DISTINCT t)): we count how many unique terminals link M and N.
Threshold Filtering (WHERE ≥ 4): this is the primary optimization. We discard all pairs that share fewer than 4 terminals immediately. This drastically reduces the number of pairs for which we need to calculate total transaction counts in the next step.
Global Count Calculation: for the few customer pairs that pass the terminal threshold, the query performs a secondary MATCH to count their total number of transactions (countM and countN).
Final Comparison: the abs(countM - countN) ≤ 2 clause applies the final project constraint, ensuring the customers have a nearly identical volume of activity.

5.3 Outlier Detection via Temporal Analysis (Query B)

The objective of this operation is to detect anomalous spending behavior at the terminal level. As defined in the project specifications, the system must analyze transactions from the "current quarter" and flag them as "potential outliers" if their value exceeds the median transaction value of the "previous quarter" by more than 30%.
This query requires a comparative analysis between two distinct time windows. Unlike a simple filtering operation, this involves an aggregation step (calculating the median) followed by a row-by-row comparison. To implement this effectively across the different synthetic datasets (Small, Medium, and Large), the Python script (*QueryB.py*) defines specific date ranges for the "Previous" and "Current" quarters based on the simulation duration.

- **Time Windowing**: for the standard analysis, we defined the "Previous Quarter" as the baseline period (e.g., Jan 1 – Mar 31) and the "Current Quarter" as the observation period (e.g., Apr 1 – Jun 30).
- **Date Parsing**: the dataset stores dates as strings (e.g., "2025-01-01 12:00:00"). To perform range queries, the Cypher query dynamically converts these strings into comparable Neo4j Date objects using date(datetime(replace(...))).

The following Cypher statement was implemented to execute this analysis in a single pass:

```
MATCH (t:Terminal)<-[:OCCURRED_AT]-(tx_prev:Transaction)
WHERE date(datetime(replace(tx_prev.date, ' ', 'T'))) >=
    date($start_prev)
  AND date(datetime(replace(tx_prev.date, ' ', 'T'))) <=
      date($end_prev)
WITH t, percentileCont(tx_prev.amount, 0.5) as median_prev
MATCH (t)<-[:OCCURRED_AT]-(tx_curr:Transaction)
WHERE date(datetime(replace(tx_curr.date, ' ', 'T'))) >=
    date($start_curr)
  AND date(datetime(replace(tx_curr.date, ' ', 'T'))) <=
      date($end_curr)
AND tx_curr.amount > (1.3 * median_prev)
RETURN
    t.id as Terminal_ID,
    median_prev as Prev_Median_Amount,
    tx_curr.id as Outlier_Tx_ID,
    tx_curr.amount as Outlier_Amount,
    tx_curr.date as Tx_Date,
    "potential outlier" as Label
ORDER BY t.id, tx_curr.amount DESC
```

Listing 2: Cypher query B for detecting outlier

Detailed Step-by-Step explanation:
Baseline Phase (MATCH ... tx_prev): The query first retrieves all transactions occurring at a terminal within the defined "previous" date range.
Median Calculation (percentileCont): We use the percentileCont(..., 0.5) function to calculate the precise median of transaction amounts for that period. This value, median_prev, serves as the benchmark for normal behavior at that specific terminal.
Comparison Phase (MATCH ... tx_curr): The query then retrieves a second set of transactions for the same terminal, but strictly within the "current" date range.
Outlier Thresholding: The core business logic is applied in the WHERE clause: tx_curr.amount > (1.3 * median_prev). This filters out any transaction that is not significantly higher than the historical baseline.
Result Labeling: The matching transactions are returned with the label "potential outlier," along with the contextual data (Terminal ID, Original Median, and Outlier Amount) for verification

5.4 Co-Customer Network Analysis (Query C)

The project requires determining the "co-customer network of degree k" ($CN_k(u)$) for a specific user u. Specifically, we are asked to compute $CN_3(u)$. In the context of our graph model, a user y belongs to $CN_3(u)$ if there is a path connecting them via shared terminals:

$$u_1 \rightarrow t_1 \rightarrow u_2 \rightarrow t_2 \rightarrow u_3$$

Where u_1 is the starting customer, u_2 are intermediaries who share a terminal with u_1, and u_3 are the targets who share a terminal with u_2.
The computation of this network involves a deep graph traversal. As the dataset size increases, the number of potential paths grows exponentially (a problem known as "combinatorial explosion"). Therefore, we implemented two different versions of this query depending on the dataset scale.

5.4.1 Standard Implementation (Small Medium Datasets)

For the 50MB and 100MB datasets, the graph density allows for an exact computation of the entire network. The script *QueryC.py* executes a full traversal to find every single unique customer at "Level 3".

```
MATCH (u1:Customer {id: $uid})

MATCH (u1)-[:PERFORMED]->(:Transaction)-[:OCCURRED_AT]->(t1:
    Terminal)<-[:OCCURRED_AT]-(:Transaction)<-[:PERFORMED]-(
    u2:Customer)
WHERE u2 <> u1

MATCH (u2)-[:PERFORMED]->(:Transaction)-[:OCCURRED_AT]->(t2:
    Terminal)<-[:OCCURRED_AT]-(:Transaction)<-[:PERFORMED]-(
    u3:Customer)
WHERE u3 <> u1 AND u3 <> u2

RETURN
    count(DISTINCT u3) as network_size,
    collect(DISTINCT u3.id)[0..10] as sample_members
```

Listing 3: Cypher query C for Co-customer Network Analysis

Logic: The query hops from the start user (u_1) to all terminals they used (t_1), then finds all other users (u_2) who used those terminals. From this set of intermediate users, it repeats the process to find the final set (u_3).
Constraint: We strictly enforce WHERE u3 <> u1 AND u3 <> u2 to ensure we are identifying new connections and not looping back to the start.

5.4.2 Optimized Implementation (Large Dataset)

For the Large dataset (200MB, $\tilde{3}67$ days of transactions), the "branching factor" (the average number of co-customers per terminal) creates a computational bottleneck. A single user might connect to hundreds of u_2 users, who in turn connect to thousands of u_3 users, resulting in millions of paths that exhaust the available RAM. To address this, we developed a "Light" version of the query (*QueryCRidotta.py*). This version introduces a sampling limit on the intermediate layer (u_2).

```
MATCH (u1:Customer {id: $uid})
MATCH (u1)-[:PERFORMED]->(:Transaction)-[:OCCURRED_AT]->(t1:
    Terminal)<-[:OCCURRED_AT]-(:Transaction)<-[:PERFORMED]-(
    u2:Customer)
WHERE u2 <> u1

// --- OPTIMIZATION: LIMIT INTERMEDIARIES ---
// We limit the expansion to the first 100 distinct
    intermediaries
// to prevent memory overflow on the Large dataset.
WITH u1, u2 LIMIT 100

MATCH (u2)-[:PERFORMED]->(:Transaction)-[:OCCURRED_AT]->(t2:
    Terminal)<-[:OCCURRED_AT]-(:Transaction)<-[:PERFORMED]-(
    u3:Customer)
WHERE u3 <> u1 AND u3 <> u2

RETURN
    count(DISTINCT u3) as network_size,
    collect(DISTINCT u3.id)[0..10] as sample_members
```

Listing 4: Cypher query C for Co-customer Network Analysis (light version)

The LIMIT 100 Clause: instead of expanding the network through every possible intermediate user, we select a subset of 100 intermediaries (u_2). This provides an approximate view of the user's network reach without crashing the database engine. In a real-world fraud detection scenario, analyzing a sample of the network is often sufficient to detect suspicious clusters without requiring a full exhaustive search.

5.5 Weekly Outlier Analysis (Query E)

The final operational requirement is to generate a summary table that analyzes transaction volume and fraud patterns across the days of the week. Specifically, the system must calculate the total number of transactions and the proportion of transactions flagged as outliers for each day (Monday through Sunday). Crucially, the definition of an "outlier" must remain consistent with the logic established in Query B (transactions exceeding the previous quarter's median by >30%).
This query performs an OLAP-style (Online Analytical Processing) operation. It does not merely filter data; it re-evaluates the outlier condition dynamically across the entire dataset and then aggregates the results by a temporal dimension.
The Python script *QueryE.py* orchestrates this by:

1. **Re-establishing the Baseline**: it calculates the historical median for every terminal (using the "Previous Quarter" window).

2. **Temporal Extraction**: it processes the "Current Quarter" transactions, converting their timestamp strings into Neo4j Date objects to extract the Day of the Week (where 1 = Monday, 7 = Sunday).

3. **Boolean Flagging**: it applies the outlier formula to every transaction, creating a temporary boolean flag (is_outlier).

4. **Aggregation**: it groups the results by day, counting the total volume versus the volume of outliers.

The following advanced Cypher query allows us to perform this multi-step analysis in a single execution:

```
MATCH (t:Terminal)<-[:OCCURRED_AT]-(tx_prev:Transaction)
WHERE date(datetime(replace(tx_prev.date, ' ', 'T'))) >=
    date($start_prev)
  AND date(datetime(replace(tx_prev.date, ' ', 'T'))) <=
      date($end_prev)

WITH t, percentileCont(tx_prev.amount, 0.5) as median_prev

MATCH (t)<-[:OCCURRED_AT]-(tx_curr:Transaction)
WHERE date(datetime(replace(tx_curr.date, ' ', 'T'))) >=
    date($start_curr)
  AND date(datetime(replace(tx_curr.date, ' ', 'T'))) <=
      date($end_curr)

WITH t, median_prev, tx_curr,
     datetime(replace(tx_curr.date, ' ', 'T')).dayOfWeek as
         day_number

WITH day_number, tx_curr,
     (tx_curr.amount > (1.3 * median_prev)) AS is_outlier

RETURN
    day_number,
    count(tx_curr) AS total_transactions,
    sum(CASE WHEN is_outlier THEN 1 ELSE 0 END) AS
        outlier_count
ORDER BY day_number
```

Listing 5: Cypher query E for Weekly Outlier Analysis

Detailed Step-by-Step Explanation:
Context Carry-over: the query begins exactly like Query B, matching tx_prev to calculate median_prev. This ensures that our summary statistics are based on the accurate historical behavior of each specific terminal, rather than a global average.

datetime().dayOfWeek: we utilize Neo4j's temporal functions to extract the weekday from the transaction string. This allows us to bin the data into 7 distinct groups.
Conditional Summation (CASE WHEN): since Cypher does not have a direct "count if" function, we use a CASE statement. If is_outlier is true, we add 1; otherwise, we add 0. This sums up the outlier_count.
Post-Processing: the raw results are returned to the Python script, where Pandas is used to calculate the final percentage (outlier_count / total_transactions * 100) and map the numeric day IDs (1–7) to readable names (Monday–Sunday) for the final report table.

6 Performance Analysis

This chapter presents the experimental results obtained by executing the project workflow on the three synthetic datasets defined in Chapter 4. The primary objective is to evaluate how the system's performance scales as the data volume increases from 50MB (Small) to 100MB (Medium) and 200MB (Large).
The following charts provide a high-level summary of the total execution times across the three scenarios.

Operation	**50MB dataset**	**100MB dataset**	**200MB dataset**
Dataset Generation	83.35 s	162.75 s	528.70 s
Neo4j Population	136.13 s	253.29 s	484.47 s
Update Transactions (d.i)	35.46 s	82.20 s	259.63 s
Create Collaborators (d.ii)	12.84 s	32.47 s	71.37 s
Query A	143.84 s	378.68 s	1195.24 s
Query B	23.59 s	50.31 s	64.69 s
Query C	57.06 s	647.78 s – 28.67 s	/ – 79.41 s
Query E	20.14 s	37.99 s	66.24 s

Table 2: Comparison of execution times across Small, Medium, Large datasets.

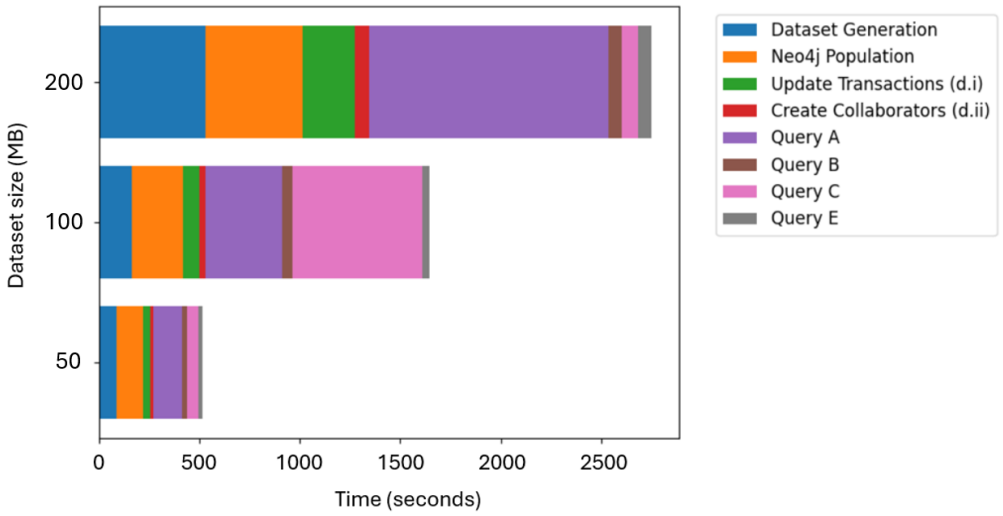


Figure 5: Cumulative execution time breakdown

As shown in the figures, most operations exhibit a linear growth trend proportional to the dataset size. However, specific graph-intensive operations (such as the creation of "Frequent Collaborators" and the "Co-customer Network" analysis) show exponential growth, highlighting the computational complexity of deep traversals in denser graphs.

6.1 Dataset Generation

The time required for synthetic data generation increases significantly as the target dataset size grows, as shown by the following measured execution times:

- Small Dataset (50MB): 83.35 s $\sim$ 1 min 23 s
- Medium Dataset (100MB): 162.75 s $\sim$ 2 min 43 s
- Large Dataset (200MB): 528.70 s $\sim$ 8 min 49 s

Although the number of customers (5000) and terminals (10 000) is kept constant across all experiments, the dataset size is determined by the number of generated transactions, which is increased by simulating longer time spans.

- The **small dataset (50 MB)**, obtained by simulating 97 days and generating about 0.93 million transactions, required 83.35 s.
- The **medium dataset (100 MB)**, corresponding to 187 days and approximately 1.79 million transactions, took 162.75 s, showing an almost linear increase with respect to the dataset size.
- However, the **large dataset (200 MB)**, generated over 367 days with about 3.52 million transactions, required 528.70 s, exhibiting a superlinear growth in generation time. This behavior is attributable to the cumulative overhead of producing and storing a much larger volume of synthetic transactions, including increased memory usage, more frequent disk I/O operations, and internal data structure management during generation.

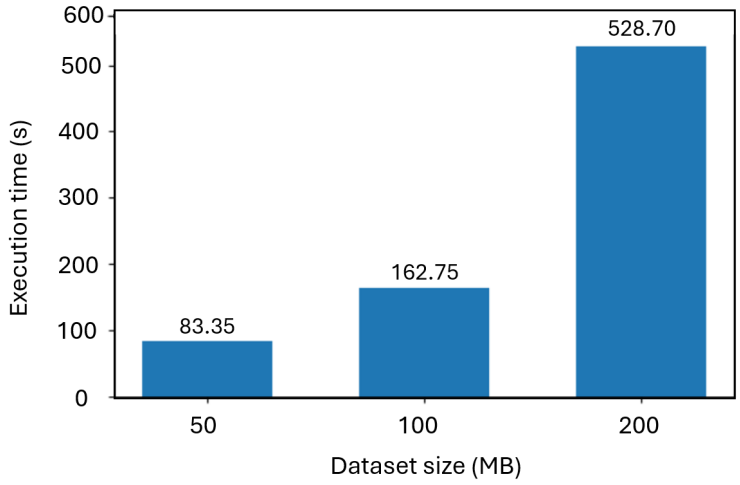


Figure 6: Executing time of dataset generation

The bar plot in Figure 6 highlights this trend clearly, emphasizing that while synthetic data generation scales mainly with the number of transactions, larger

datasets introduce additional system-level costs that make the generation process increasingly expensive as the dataset size grows.

6.2 Neo4j Population

The time required to populate the Neo4j database also increases consistently with the dataset size. The obtained population times for the three datasets are:

- Small Dataset (50MB): 136.13 s $\sim$ 2 min 16 s
- Medium Dataset (100MB): 253.29 s $\sim$ 4 min 13 s
- Large Dataset (200MB): 484.47 s $\sim$ 8 min 4 s

For the small dataset **50 MB**, importing the CSV files containing customers, terminals, and transactions into Neo4j required 136.13 s. When the dataset size was increased to **100 MB**, the population time rose to 253.29 s, while the largest dataset **200 MB** required 484.47 s to be fully loaded.
Since the loading procedure was identical for all datasets, consisting solely of importing CSV files and creating the corresponding nodes and relationships, these results primarily reflect the growing number of records, especially transactions, that Neo4j must parse, validate, and persist on disk. The near-linear scaling observed between the medium and large datasets suggests that Neo4j's bulk import and transaction handling mechanisms manage increasing volumes of data efficiently, although larger datasets inevitably incur higher costs due to index updates, relationship creation, and disk I/O operations.
Overall, the bar plot in Figure 7 highlights that database population time scales mainly with the amount of data being ingested, confirming that transaction volume is the dominant factor affecting the Neo4j loading performance.

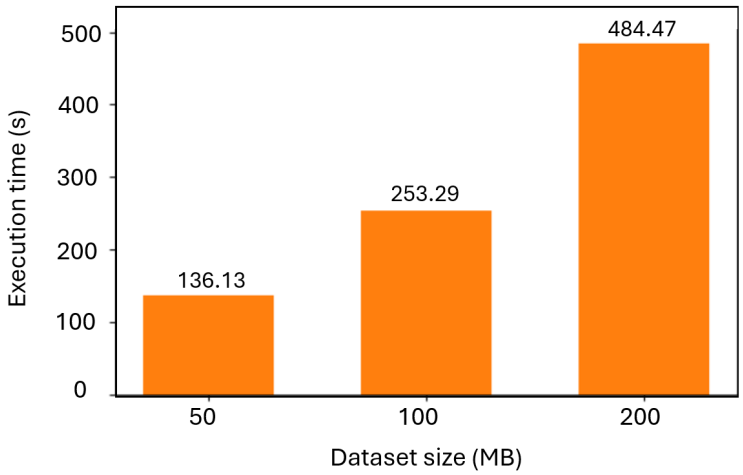


Figure 7: Executing time of Neo4j population

6.3 Update Transactions (d.i)

The execution times for updating the Neo4j database by adding additional properties to existing nodes and relationships show a clear dependence on the dataset size:

- Small Dataset (50MB): 35.46 s
- Medium Dataset (100MB): 82.20 s $\sim$ 1 min 22 s
- Large Dataset (200MB): 259.63 s $\sim$ 4 min 20 s

For the small dataset **50 MB**, the update operation required 35.46 s, while the medium dataset **100 MB** took 82.20 s, corresponding to about 1 min and 22 s. In the case of the large dataset **200 MB**, the update time increased substantially to 259.63 s (approximately 4 min and 20 s). Since this operation was performed after the initial data loading and involved extending each transaction with new attributes, namely the payment method, promotional flag, and customer satisfaction rating, the execution time scales mainly with the number of transaction nodes to be updated. The more-than-linear increase observed for the largest dataset can be attributed, again, to the overhead associated with writing a large number of property updates to disk, as well as Neo4j's internal transaction management and logging mechanisms, which become more expensive as the volume of updated entities grows.
Overall, the bar plot in Figure 8 highlights that even relatively simple update operations can become costly at scale, emphasizing the importance of considering transaction volume and update complexity when extending the logical model in a graph database.

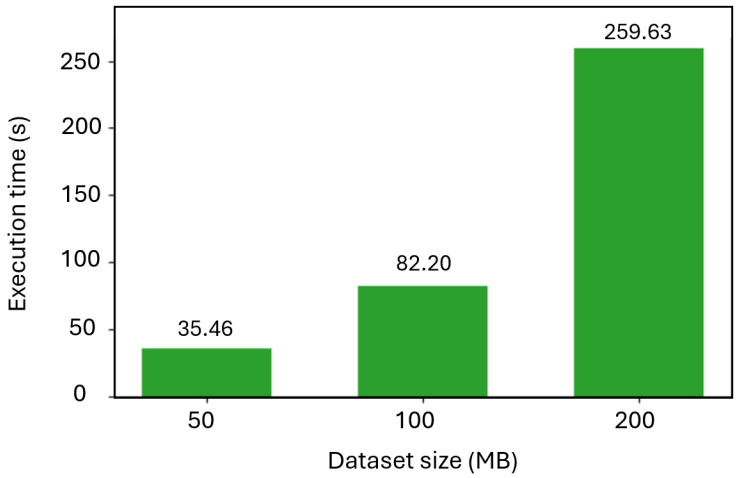


Figure 8: Executing time of transaction update

6.4 Create Collaborators (d.ii)

The execution times for creating *frequent collaborator* relationships in Neo4j increase with the dataset size:

- Small Dataset (50MB): 12.84 s
- Medium Dataset (100MB): 32.47 s
- Large Dataset (200MB): 71.37 s $\sim$ 1 min 11 s

Unlike previous operations, this step involves non-trivial graph processing: Neo4j must analyze transaction patterns to identify customers who have completed at least five transactions at the same terminal and then compute and compare their average satisfaction ratings before creating the corresponding relationships. As the dataset grows, the number of transactions per terminal and the potential customer pairings increase, leading to a higher number of aggregations, comparisons, and relationship creations.
The bar plot in Figure 9 highlights that, despite this added complexity, the growth in execution time remains manageable and linear compared to the increase in data volume.

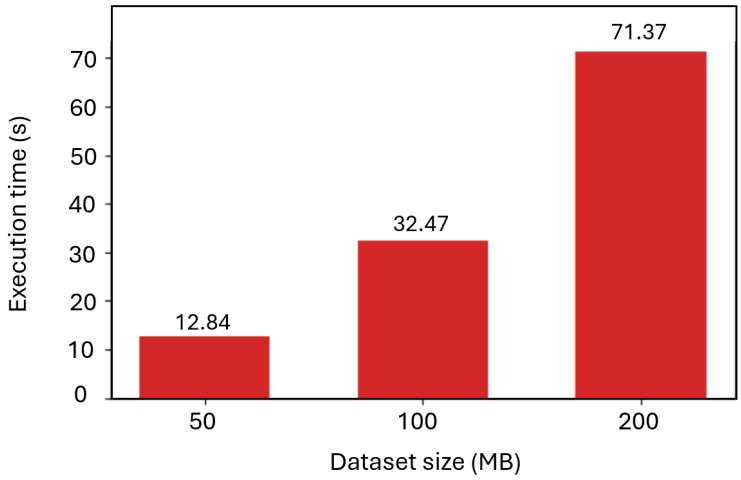


Figure 9: Executing time for creating collaborators

6.5 Query A

The query A aims to identify, for each customer M, all customers N who have made transactions at at least four common terminals and whose total number of transactions differs by no more than two. For each qualifying pair, the query returns the names of customers M and N, the number of shared terminals, and their respective transaction counts. The obtained times for the three datasets are:

- Small Dataset (50MB): 143.84 s $\sim$ 2 min 23 s
- Medium Dataset (100MB): 378.68 s $\sim$ 6 min 19 s
- Large Dataset (200MB): 1195.24 s $\sim$ 19 min 55 s

On the small dataset **50 MB**, the query required 143.84 s (approximately 2 min and 23 s). When the dataset size was increased to **100 MB**, the execution time rose significantly to 378.68 s (about 6 min and 19 s). The large dataset **200 MB** further amplified this cost, with a total execution time of 1195.24 s (nearly 20 min). This pronounced growth is mainly due to the need to compare customer behaviors across a large number of transactions and terminals.
The bar plot in Figure 10 clearly illustrates this trend, emphasizing that the query on large graphs are considerably more expensive than data loading or update operations.

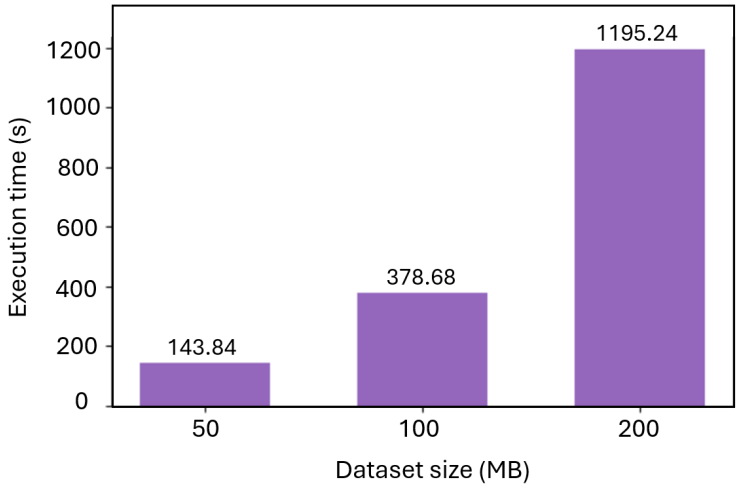


Figure 10: Executing time of query A

The following tables report sample outputs of the query executed on the three datasets, showing the identified customer pairs and the corresponding metrics:

Customer_M	Customer_N	Shared_Terminals	Tx_Count_M	Tx_Count_N
744	1743	92	356	356
1415	4106	82	313	313
3332	3542	81	350	351
1068	3360	80	343	343
2267	3570	76	341	342

Table 3: Query A results on the Small Dataset (first 5 rows)

Customer_M	Customer_N	Shared_Terminals	Tx_Count_M	Tx_Count_N
2281	2899	89	647	645
1793	3542	80	678	680
2768	4678	79	453	454
652	2890	76	545	543
782	3185	74	277	277

Table 4: Query A results on the Medium Dataset (first 5 rows)

Customer_M	Customer_N	Shared_Terminals	Tx_Count_M	Tx_Count_N
1595	2009	85	1356	1357
220	2552	82	1155	1155
3186	3702	82	736	736
624	4560	80	942	941
2610	4301	79	1274	1273

Table 5: Query A results on the Large Dataset (first 5 rows)

6.6 Query B

Query B requires, for each terminal, the identification of transactions in the current quarter whose value exceeds by more than 30% the median transaction value computed for the same terminal in the previous quarter. Transactions satisfying this condition are marked as potential outliers. The execution times for Query B are reported as follows:

- Small Dataset (50MB): 23.59 s
- Medium Dataset (100MB): 50.31 s
- Large Dataset (200MB): 64.69 s $\sim$ 1 min 5 s

On the small dataset **50 MB**, the query completed in 23.59 s and identified 25805 potential outliers. Due to the limited temporal span of this dataset (97 days), it was not possible to define two complete consecutive quarters; therefore, the computation was approximated by considering the period from 2025-01-01 to 2025-03-31 as the reference interval and the interval from 2025-04-01 to 2025-04-10 as the current quarter. For the medium dataset **100 MB**, which spans a sufficiently long time window, the query required 50.31 s and detected 332 496 outliers by comparing transactions in the previous quarter (2025-01-01 to 2025-03-31) with those in the current quarter (2025-04-01 to 2025-06-30). A similar configuration was applied to the large dataset **200 MB**, where the query completed in 64.69 s (approximately 1 min and 5 s) and identified 332 494 outliers.

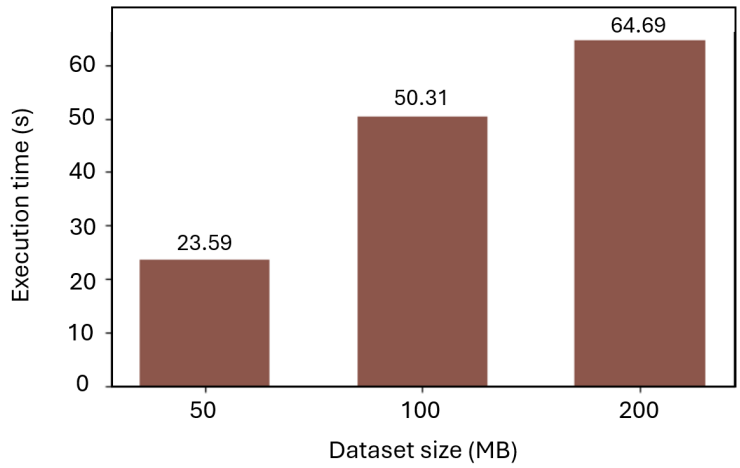


Figure 11: Executing time of query B

The observed execution times scale reasonably with the dataset size and are mainly influenced by the number of transactions per terminal involved in the

median computation and subsequent filtering. Unlike pairwise customer comparisons, this query operates independently on each terminal, relying on aggregation and value-based filtering rather than combinatorial matching. As a result, even though the number of processed transactions increases significantly, the growth in execution time remains relatively contained. The bar plot in Figure 11 highlights this behavior.
The following tables report sample outputs of the query executed on the three datasets, showing the identified customer pairs and the corresponding metrics:

Terminal_ID	Prev_Median_Amount	Outlier_Tx_ID	Outlier_Amount	Tx_Date	Label
0	54.22	917555	134.98	2025-04-06 13:54:19	potential outlier
1	67.56	919809	104.36	2025-04-06 18:00:39	potential outlier
2	54.95	887901	99.07	2025-04-03 12:48:39	potential outlier
3	43.79	877186	82.46	2025-04-02 11:20:20	potential outlier
4	48.39	905532	107.49	2025-04-05 10:29:55	potential outlier

Table 6: Query B results on the Small Dataset (first 5 rows)

Terminal_ID	Prev_Median_Amount	Outlier_Tx_ID	Outlier_Amount	Tx_Date	Label
0	54.22	1060064	209.47	2025-04-21 12:41:16	potential outlier
0	54.22	1700883	193.79	2025-06-27 10:56:18	potential outlier
0	54.22	1547740	165.70	2025-06-11 11:50:22	potential outlier
0	54.22	1675261	159.80	2025-06-24 15:28:20	potential outlier
0	54.22	1170509	159.03	2025-05-03 04:43:22	potential outlier

Table 7: Query B results on the Medium Dataset (first 5 rows)

Terminal_ID	Prev_Median_Amount	Outlier_Tx_ID	Outlier_Amount	Tx_Date	Label
0	54.22	1060064	209.47	2025-04-21 12:41:16	potential outlier
0	54.22	1700883	193.79	2025-06-27 10:56:18	potential outlier
0	54.22	1547740	165.70	2025-06-11 11:50:22	potential outlier
0	54.22	1675261	159.80	2025-06-24 15:28:20	potential outlier
0	54.22	1170509	159.03	2025-05-03 04:43:22	potential outlier

Table 8: Query B results on the Large Dataset (first 5 rows)

6.7 Query C

The Query C computes the co-customer network of degree 3, identifying all customers connected to a given user u through paths of length k via shared transaction terminals, with the constraint that no customer is repeated along the path. The obtained times for executing the query on the three datasets are:

- Small Dataset (50MB): 57.06 s
- Medium Dataset (100MB): 647.78 s $\sim$ 10 min 48 s
- Large Dataset (200MB) with LIMIT: 79.41 s $\sim$ 1 min 19 s

On the small dataset **50 MB**, the computation required 57.06 s, while the medium dataset **100 MB** took 647.78 s (approximately 10 min and 48 s). For the large dataset **200 MB**, attempting a full computation proved infeasible, as the query did not finish even after more than one hour of execution. To address this, we applied a **LIMIT 100** clause to restrict the expansion to a subset of 100 intermediate users u_2. With this restriction, the query completed in 79.41 s (approximately 1 min and 19 s).
The substantial increase in execution time from the small to medium dataset highlights the combinatorial nature of the problem: the number of possible paths grows rapidly with both the number of customers and terminals. The application of the LIMIT clause on the large dataset was necessary to obtain results in a practical timeframe, and it provides an approximate but meaningful view of the network structure.
The bar plot in Figure 12 clearly illustrates how the computational cost of network expansion queries in Neo4j can escalate dramatically with dataset size and network density.

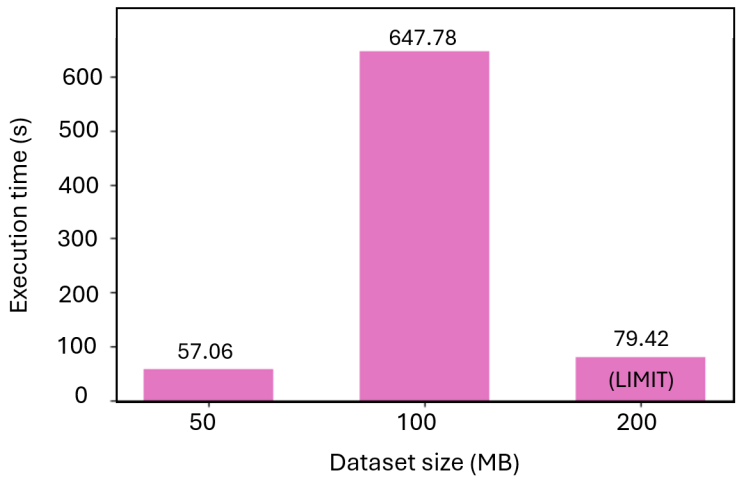


Figure 12: Executing time of query C

The first ten elements of the computed co-customer network are reported below for each dataset.

Dataset	**First 10 Nodes in CNk**
Small (50 MB, degree 3)	[143, 195, 204, 296, 410, 502, 596, 599, 732, 803]
Medium (100 MB, degree 3)	[228, 726, 1031, 1439, 1440, 1624, 1746, 1770, 1816, 1865]
Large (200 MB, degree 2, LIMIT 100)	[1540, 1631, 2210, 2499, 573, 842, 4886, 3790, 3912, 4286]

Table 9: Query C – Sample Co-Customer Network Output

6.8 Query E

The query E aggregates transaction data by day of the week and computes the proportion of transactions flagged as outliers. The obtained times for executing the query show a moderate increase with the dataset size, as illustrated as follows:

- Small Dataset (50MB): 20.14 s
- Medium Dataset (100MB): 37.99 s
- Large Dataset (200MB): 66.24 s $\sim$ 1 min 6 s

This query involves scanning all transactions in the dataset, grouping them by the day of the week, counting the total number of transactions per day, and determining the fraction of transactions that were previously identified as outliers in Query B. As the dataset size increases, the number of transactions to be aggregated grows, leading to longer execution times; however, the growth remains roughly linear because each transaction contributes independently to the counts and proportions. The bar plot in Figure 13 shows this trend.

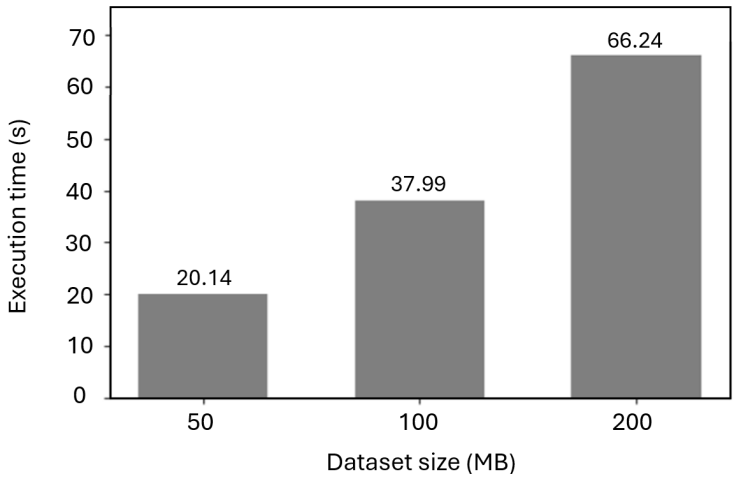


Figure 13: Executing time of query E

The resulting tables are reported below for each dataset.

day_number	day_name	total_transactions	outlier_count	proportion_outliers (%)
1	Monday	9644	3697	38.33
2	Tuesday	9789	3743	38.24
3	Wednesday	9692	3639	37.55
4	Thursday	9670	3684	38.10
5	Friday	9620	3685	38.31
6	Saturday	9542	3689	38.66
7	Sunday	9784	3668	37.49

Table 10: Query E results on the Small Dataset

day_number	day_name	total_transactions	outlier_count	proportion_outliers (%)
1	Monday	124298	47649	38.33
2	Tuesday	124447	47645	38.29
3	Wednesday	125046	47542	38.02
4	Thursday	124214	47135	37.95
5	Friday	124731	47619	38.18
6	Saturday	124490	47617	38.25
7	Sunday	124851	47289	37.88

Table 11: Query E results on the Medium Dataset

day_number	day_name	total_transactions	outlier_count	proportion_outliers (%)
1	Monday	124298	47646	38.33
2	Tuesday	124447	47645	38.29
3	Wednesday	125046	47542	38.02
4	Thursday	124214	47136	37.95
5	Friday	124731	47621	38.18
6	Saturday	124490	47617	38.25
7	Sunday	124851	47287	37.87

Table 12: Query E results on the Large Dataset

7 Conclusions

This project has been particularly interesting and valuable as it allowed us to consolidate our knowledge in modeling an application domain using NoSQL databases, with a specific focus on Neo4j. Working with graph databases provided hands-on experience in designing nodes, relationships, and properties, and in understanding how the data model affects query performance. Additionally, the project reinforced our practical skills in Python, both for generating synthetic datasets and for interacting with the database.

It was also insightful to observe how queries and operations are applied in a scenario resembling real-life situations. For instance, analyzing transactions, detecting outliers, and exploring customer networks helped us appreciate the practical challenges of working with data at scale. One important realization was the significant amount of time that operations can take when executed on (pseudo-)realistic datasets. This highlighted the importance of efficient query design, aggregation strategies, and sometimes approximations or sampling when dealing with large datasets.

Overall, the project has provided a concrete, applied understanding of NoSQL modeling, graph databases, and data analysis workflows, bridging the gap between theoretical knowledge and practical implementation. It offered a realistic view of the computational costs involved in data-intensive tasks, which is highly relevant for future work in data management and analytics.